Universidad de Antioquia

Facultad de Ingeniería

Informática II

Semestre 2026-2

UdeABooK

Informe

Cristian Muñoz

Pablo Gonzalez

Medellín, octubre de 2026

1. Introducción

UdeABook es una red social de consola hecha en C++. En este proyecto trabajamos con usuarios, amigos, solicitudes de amistad, álbumes, fotografías y mensajes. La idea no fue construir una interfaz gráfica, sino organizar bien la información y permitir que el usuario realice las operaciones que pide la guía.

Durante la ejecución, los datos se mantienen en memoria RAM. Los archivos de texto sirven para cargar la información al comenzar y guardar los cambios al salir. Para las imágenes solamente se almacenan y muestran sus rutas, sin abrir archivos PNG.

2. Análisis del problema y propuesta de solución

El principal problema fue manejar muchas relaciones entre usuarios sin duplicar objetos ni gastar memoria de forma innecesaria. También debíamos respetar condiciones como no enviar solicitudes a usuarios que ya son amigos, no mostrar álbumes privados y no permitir mensajes de más de 512 caracteres.

Organizamos el programa en varias clases con tareas separadas. RedSocial coordina las operaciones; Usuario guarda la información de cada cuenta; Album y Foto representan el contenido de los perfiles; Mensaje guarda la comunicación; Fecha valida y compara las fechas; y Sugerencia representa temporalmente los candidatos a amistad.

Para leer y escribir los TXT usamos Archivo. La clase Recursos, junto con RegistroLocal, sirve para llevar las mediciones de iteraciones y memoria. Esta división facilita cambiar una parte sin tener que volver a escribir todo el programa.

En la solución final decidimos guardar las amistades en una matriz triangular comprimida en bits, en vez de tener dos listas de amigos para cada relación. Como una amistad es mutua, no necesitamos guardar dos veces la misma pareja de usuarios. Las solicitudes recibidas usan índices empaquetados y los álbumes y fotografías se almacenan en arreglos dinámicos propios.

3. Organización del programa y clases

RedSocial es la clase principal del sistema. Tiene el arreglo de usuarios, la matriz de amistades, los mensajes ya cargados y la lista de mensajes nuevos. También incluye los métodos que usan las opciones del menú y el manejo de los archivos.

Usuario contiene nombre, correo, contraseña, ciudad, país, fechas de nacimiento e inscripción y ruta de foto de perfil. Además, administra las solicitudes de amistad que ha recibido y sus álbumes. Album guarda nombre, ruta, privacidad, propietario y fotografías. Foto conserva la ruta, fecha de captura, tamaño, descripción y cantidad de likes.

Mensaje tiene el contenido, el emisor, el receptor, la fecha y el estado de lectura. Sugerencia contiene un usuario candidato y su número de amigos en común. NodoMensaje se usa para incorporar mensajes nuevos antes de escribirlos de nuevo en el archivo general.

Archivo trabaja con las líneas de texto, mientras que Recursos lleva las métricas. RegistroLocal ayuda a sumar y descontar la memoria de las variables locales cuando entran y salen de alcance. Interfaz no es una clase: es un módulo con una función que pide y valida opciones numéricas.

El diagrama actualizado se incluye al final del informe. Conservamos las relaciones principales y las cardinalidades del preinforme: una RedSocial administra uno o más usuarios y cero o más mensajes;

cada usuario puede tener cero o más álbumes y cada álbum cero o más fotos. También se muestran las relaciones con fechas, mensajes y las clases de apoyo que aparecieron en la implementación.

4. Formato de los archivos

Los datos se manejan principalmente en tres archivos: usuarios.txt, albumes.txt y mensajes.txt. Elegimos texto separado por comas para poder leer los campos con nuestras propias funciones. Los nombres de usuario permiten encontrar a qué persona pertenece cada registro.

usuarios.txt: cada usuario ocupa tres líneas. La primera tiene nombre, correo, contraseña, ciudad, país, fecha de nacimiento, fecha de inscripción y ruta de perfil. La segunda contiene los nombres de sus amigos separados por comas, y la tercera contiene los nombres de quienes le enviaron solicitudes. Si alguna lista está vacía, se deja su línea en blanco.

albumes.txt: cada álbum ocupa dos líneas. La primera guarda el nombre del propietario, nombre del álbum, ruta, privacidad y cantidad de fotos. La segunda guarda los datos de sus fotos; para cada una se incluyen ruta, fecha de captura, tamaño, likes y descripción. Se utiliza la cantidad de fotos para saber cuántos registros se deben recuperar.

mensajes.txt: cada línea corresponde a un mensaje y guarda receptor, emisor, fecha de envío, estado de lectura (0 o 1) y contenido. El texto se toma después de los primeros campos para que se puedan conservar comas en el mensaje.

Las fechas se escriben como AAAA-MM-DD. El programa lee los archivos al inicio, crea los objetos y después relaciona los nombres con las posiciones de los usuarios en memoria. Al finalizar escribe los valores actuales, no las direcciones de los punteros.

El formato exige cuidado con las comas y los saltos de línea en los campos. Las validaciones rechazan entradas incompatibles; por eso los archivos de prueba deben respetar el formato establecido.

5. Desarrollo de las funcionalidades

5.1. Carga y actualización de datos

Al iniciar, se leen los usuarios y se ordenan por nombre cuando hace falta. Después se construyen las amistades y se cargan las solicitudes; luego se incorporan los álbumes, las fotos y los mensajes. Este orden evita intentar utilizar un usuario que todavía no ha sido creado.

Para el guardado se actualizan las relaciones de amistad, las solicitudes, los likes y los mensajes. Estas operaciones se hacen al cerrar la sesión y no aparecen como una opción normal de modificación de datos en el menú.

5.2. Ingreso y visualización de mejores amigos

El usuario escribe su nombre y contraseña. El programa encuentra su cuenta mediante búsqueda binaria y comprueba la contraseña. Si no coincide, permite intentarlo de nuevo.

Después del ingreso se muestran hasta tres rutas de fotos de perfil de sus amigos. La elección es aleatoria y ponderada según los amigos que tienen en común. Se usa una selección sin repetir candidatos dentro del mismo grupo. El menú permite actualizar los perfiles una sola vez por sesión.

5.3. Administrar amistades

El menú permite enviar solicitudes, aceptar las recibidas y rechazarlas. Antes de enviar se comprueba que el destinatario exista, que no sea el mismo usuario, que no exista una amistad y que tampoco haya una solicitud pendiente en cualquiera de los dos sentidos.

Al aceptar se registra la relación de amistad en la matriz de bits y se retira la solicitud recibida. Si el usuario decide rechazarla, solo se elimina la solicitud. Así se evita crear relaciones que no hayan sido aceptadas.

5.4. Visitar el perfil de un amigo

Para visitar un perfil se escribe el nombre de otro usuario. RedSocial comprueba primero que sean amigos. Después muestra únicamente los álbumes que tengan privacidad pública o para amigos; los privados no aparecen en esa lista.

En el carrusel se muestra la ruta de la fotografía y el número de likes. Las opciones permiten avanzar, retroceder, dar o quitar like y salir. El recorrido es circular: después de la última fotografía vuelve a la primera, y desde la primera se puede retroceder a la última.

5.5. Ver sugerencias de amistad

El programa pide una cantidad K entre 1 y 20. Revisa los usuarios que no son amigos del usuario activo y cuenta cuántas amistades tienen en común. Con esos datos conserva las mejores K sugerencias ordenadas de mayor a menor.

La lista incluye el nombre del candidato y la cantidad de amigos en común. Desde ese resultado se puede seleccionar a una persona para enviarle una solicitud, usando las mismas validaciones de la administración de amistades.

5.6. Administrar mensajes

Solo se pueden enviar mensajes a amigos. El texto debe tener contenido y como máximo 512 caracteres. El mensaje guarda su fecha y comienza con estado no leído.

Para leer nuevos mensajes, el programa busca los recibidos por el usuario que no hayan sido leídos, los presenta en orden de fecha y los marca como leídos. En la búsqueda por rango de fechas se validan ambas fechas y se incluyen los mensajes recibidos desde el día inicial hasta el final, incluyendo esos dos días.

5.7. Medición del consumo de recursos

Después de cada operación se muestran el número de iteraciones y la memoria contabilizada. Recursos lleva el contador de iteraciones, las llamadas a funciones externas que se registran y el valor de memoria actual y máximo observado.

Para la memoria se tienen en cuenta los tamaños de objetos y variables locales y las reservas dinámicas que el programa registra. RegistroLocal permite descontar las variables cuando salen de su alcance.

Hay una limitación importante: la implementación reporta la memoria registrada por el programa, pero no incluye la memoria interna de algunas librerías ni los costos del asignador. Por eso los valores se deben interpretar como la medición implementada y no como el consumo total exacto del proceso que muestra el sistema operativo.

6. Algoritmos y decisiones importantes

6.1. Amistades en bits

Con N usuarios podrían existir N(N-1)/2 amistades distintas, porque no se cuenta la relación de una persona consigo misma y Ana-Pablo es la misma relación que Pablo-Ana. Por esa razón guardamos solo la parte superior de una matriz de adyacencia, usando un bit por pareja.

Cuando se consulta una amistad, primero se ordenan los dos índices, se calcula la posición lineal que corresponde a esa pareja y se lee el bit. Aceptar una solicitud activa ese bit. De esta manera no necesitamos una matriz completa de enteros ni dos arreglos separados de amigos.

6.2. Ordenamiento de usuarios y mensajes

Los usuarios se ordenan por nombre usando HeapSort cuando el archivo no llega ordenado. Esto permite aplicar luego búsqueda binaria sin mantener un segundo arreglo completo de usuarios.

Los mensajes iniciales también pueden ordenarse con HeapSort. Los mensajes creados durante la sesión se guardan aparte mediante nodos enlazados y se organizan antes del guardado. Finalmente, se combinan con los mensajes anteriores para escribir el archivo ordenado.

6.3. Cálculo de sugerencias

Se crea un arreglo temporal con los índices de los amigos del usuario. Después se recorren los candidatos que no son amigos y se cuenta cuántas relaciones comparten con ese arreglo. Para ahorrar memoria, se conserva una lista de tamaño K en vez de guardar todos los candidatos para ordenarlos al final.

Cada sugerencia lleva una referencia al usuario y su cantidad de coincidencias. Cuando aparece un candidato mejor, se ubica en su posición y se desplazan los elementos necesarios de la lista.

6.4. Selección aleatoria de perfiles

Para los amigos que tienen hasta 10 conocidos en común se utiliza el peso 2; para 11 a 20, el peso 5; y para más de 20, el peso 7. Son pesos equivalentes a 1, 2,5 y 3,5, pero permiten hacer el sorteo con números enteros.

Se suman los pesos disponibles y se sortea una posición de esa suma. El amigo elegido se muestra y su peso se retira para que no salga repetido dentro del mismo grupo. La actualización se controla con una variable booleana en el menú y solo puede hacerse una vez.

6.5. Privacidad, fechas y validaciones

Antes de mostrar un álbum se revisa tanto la amistad como la privacidad. Las fechas se manejan mediante objetos Fecha, que comprueban valores de día, mes y año y permiten comparar las fechas de los mensajes.

Las entradas numéricas del menú se validan para evitar índices fuera del arreglo. También se revisan usuarios inexistentes, solicitudes repetidas y mensajes vacíos o demasiado largos antes de modificar cualquier dato.

7. Dificultades y cambios durante el desarrollo

Una de las primeras decisiones fue cómo almacenar las amistades. En el preinforme se propusieron arreglos de punteros a amigos, pero después vimos que una matriz triangular en bits nos permitía guardar cada amistad una sola vez y ocupar menos espacio.

También cambió la manera de organizar los archivos. Al principio Recursos se usaba para leer y escribir líneas. En la implementación final se creó Archivo para esa tarea y dejamos Recursos para las mediciones, porque eran responsabilidades diferentes.

Otro aspecto que exigió cuidado fue trabajar con memoria dinámica. Al copiar usuarios o álbumes se deben copiar correctamente los arreglos que les pertenecen y conservar las referencias a los objetos adecuados. Esto es importante para evitar perder fotografías o dejar punteros con propietarios equivocados.

Para los mensajes también hubo cambios. En lugar de mover y ampliar todo el arreglo ordenado en cada envío, se incorporó NodoMensaje para guardar mensajes nuevos durante la ejecución y mezclarlos con los anteriores al terminar.

Finalmente, tuvimos que organizar el trabajo de dos ramas de Git. Las clases evolucionaron mientras se desarrollaban las funcionalidades, por lo que fue necesario integrar cambios y mantener compatibles los archivos de cabecera y sus implementaciones.

8. Evolución de la solución

En el primer diseño planteamos las clases principales y las cardinalidades entre ellas. Durante la implementación se mantuvieron esas relaciones básicas, pero cambiaron algunas estructuras internas. RedSocial pasó a manejar las amistades en bits; Usuario ahora guarda solicitudes mediante índices comprimidos y álbumes dinámicos; y Foto conserva un contador de likes, sin lista de usuarios que dieron like, porque la guía no exige validar un único like por persona.

Se agregaron Archivo para la persistencia, NodoMensaje para administrar mensajes nuevos y RegistroLocal para completar las mediciones. Sugerencia se mantuvo como un objeto temporal generado al consultar recomendaciones. El módulo Interfaz se utiliza para validar opciones de la consola, sin convertirse en otra clase del sistema.

En conjunto, la solución pasó de un diseño general a una implementación modular con archivos .h y .cpp, constructores, destructores, copias, operadores sobrecargados, referencias y memoria dinámica, sin usar contenedores de la STL.

9. Verificación y pruebas

El código incorpora validaciones para los casos más comunes: usuarios inexistentes, amistades repetidas, solicitudes pendientes, álbumes sin fotos, entradas numéricas incorrectas y fechas inválidas. El menú permite ejecutar las funcionalidades y mostrar las métricas correspondientes.

Antes de entregar es necesario dejar registradas las pruebas de ejecución completa con los archivos definitivos y especialmente con al menos 1000 usuarios. La guía pide demostrar 12 sugerencias, una búsqueda de mensajes de por lo menos seis semanas y un perfil con dos álbumes de cinco fotos cada uno. No se incluyen aquí valores de memoria o iteraciones porque aún no se cuenta con resultados verificados de esas ejecuciones.

10. Conclusiones

El desarrollo de UdeABook nos permitió aplicar programación orientada a objetos para un problema donde varias clases comparten información y se relacionan entre sí. Separar las tareas ayudó a organizar el proyecto y a que el menú principal no tuviera que contener todos los algoritmos.

La estructura de amistades en bits fue una de las decisiones más importantes para reducir memoria. Los arreglos dinámicos se usaron para álbumes, fotos y registros, mientras que los nodos se reservaron para los mensajes nuevos que llegan durante una sesión.

Consideramos que la solución implementa las funcionalidades solicitadas a nivel de código. Para cerrar el trabajo correctamente todavía se deben comprobar las pruebas con el conjunto grande de datos, revisar sus métricas y presentar las evidencias en el video.

11. Diagrama de clases

El diagrama siguiente resume las clases principales, sus atributos, métodos y cardinalidades. El diagrama detallado, con todos los atributos y métodos, se entrega como un PDF independiente en la carpeta informes del repositorio: Diagrama_completo_UdeABook.pdf.

UdeABook | Diagrama resumido

